

Quality Assurance in SW- Development (Practicals)

Assignment 2 “Tutorial” – ScalaCheck, TorXakis

Summer 2025

What we have learned (and desired) so far:

"Don't write test cases,
generate them!"

(John Hughes)

Your task:

You are given an Actor System and a Message Board in Java and its Model in Scala, as well as some simple functions in Java and Scala

In your groups:

- Write properties in ScalaCheck for the simple Scala functions

- Write ScalaCheck properties for the Message Board

- Complete the Model specification in Scala for the Message Board

- Detect Mutants from your Scala model with Python

How can one do this?

Setup Scala SBT in IntelliJ

Very similar to setting up SDK for Java

No Scala SDK in module

Setup Scala SDK

OFF

1

package at.tugraz.sai.qs2025

2

3

import ...

7

sebastian.albu

8

class MessageBoardSpecificationTest extends TestCase {

9

sebastian.albu

10

def testMessageBoardModelBased(): Unit = {

11

val params: Parameters => Parameters = p => p

12

.withTestCallback(ConsoleReporter(2))

.withMinSuccessfulTests(300)



Setup Scala SBT in IntelliJ

Very similar to setting up SDK for Java (Gradle detects it)

No Scala SDK in module [Setup Scala SDK](#) OFF

```

1 package at.tugraz.sai.qs2025
2
3 import ...
7
8 class MessageBoardSpecificationTest extends TestCase {
9     def testMessageBoardModelBased(): Unit = {
10         val params: Parameters => Parameters = p => p
11             .withTestCallback(ConsoleReporter(2))
12             .withMinSuccessfulTests(300)
13         val result: Boolean = Test.check(MessageBoardSpecification.property())(params)
    
```

sebastian.albu

Add Scala Support

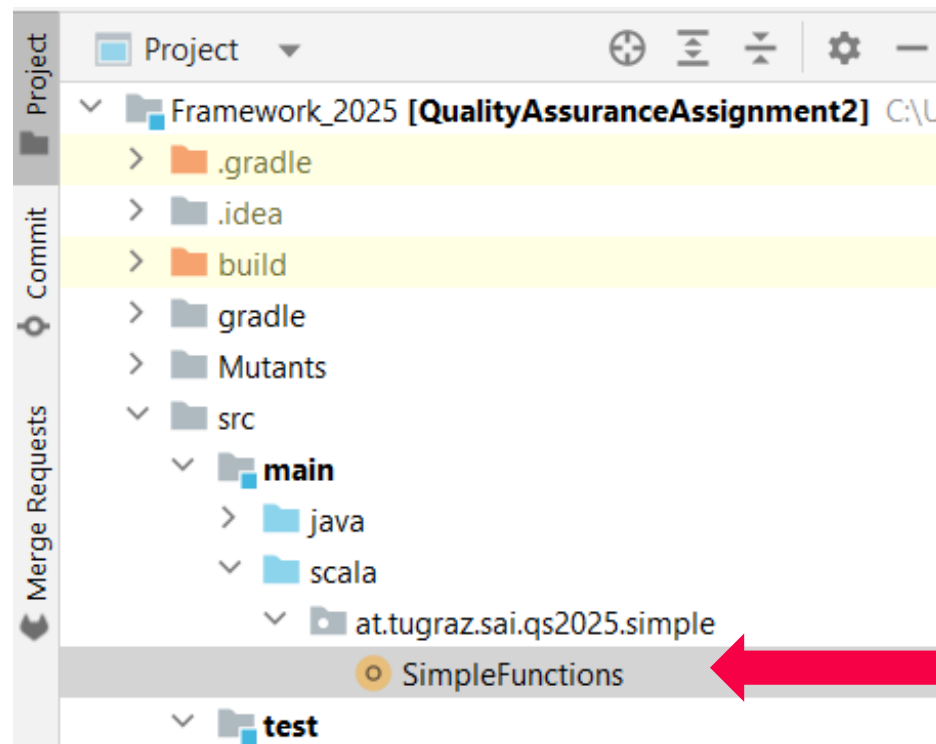
Use library:

Gradle: org.scala-lang:scala-library:2.13.1 library will be used



SimpleFunctions ScalaCheck properties

Write ScalaCheck properties for each function defined in SimpleFunctions.scala



SimpleFunctions ScalaCheck properties

Write ScalaCheck properties for each function defined in SimpleFunctions.scala

Example function:

```

37 // -----
38 // Maximum
39
40 /**
41  * Get largest integer/maximum of list
42  *
43  * @param xs a non-empty list
44  * @return largest integer
45  */
46 // sebastian.albu
47 def max(xs: List[Int]): Int = xs match {
48   case a :: as => max(a, as)
49 }

```

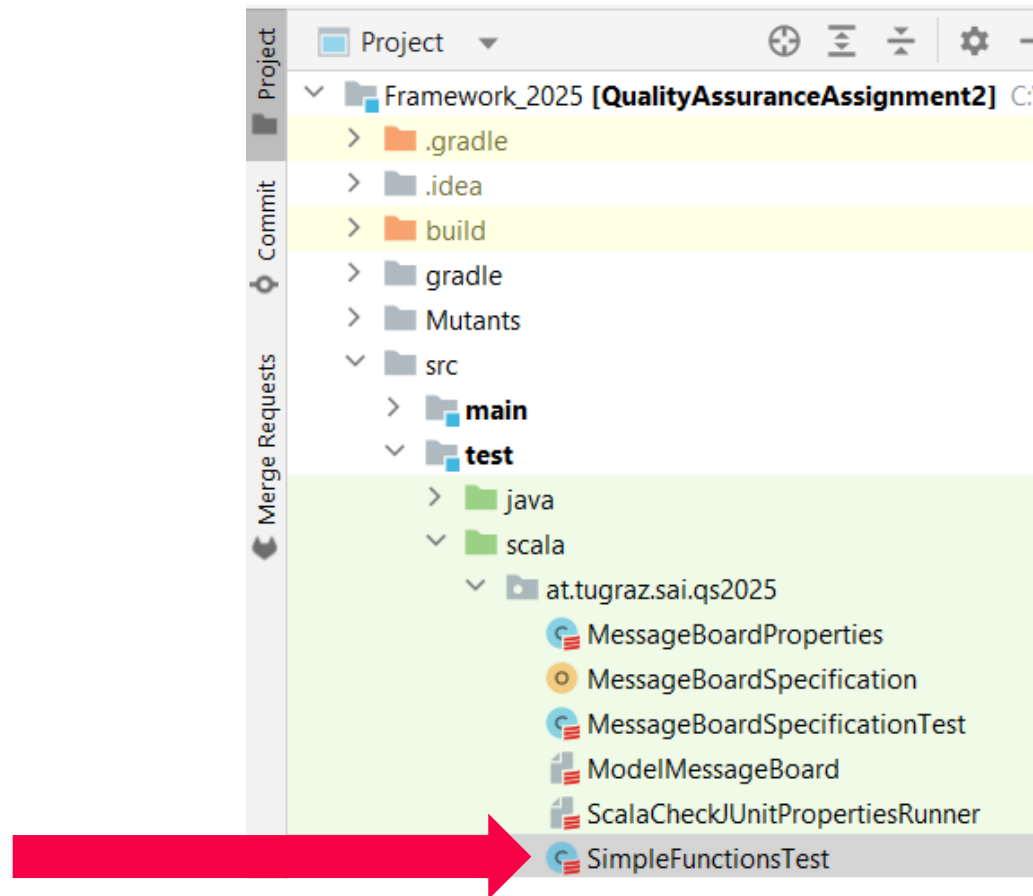
```

50 /**
51  * Tail recursive helper function for maximum.
52  * Gives maximum of integer x and list xs
53  *
54  * @param x a integer
55  * @param xs a list
56  * @return maximum of x and xs
57  */
58 // sebastian.albu
59 @scala.annotation.tailrec
60 private def max(x: Int, xs: List[Int]): Int = xs match {
61   case Nil => x
62   case a :: as =>
63     if (x >= a) max(x, as)
64     else max(a, as)
65 }

```

SimpleFunctions ScalaCheck properties

Write the properties in the SimpleFunctionsTest.scala file



SimpleFunctions ScalaCheck properties

Example ScalaCheck property for `max(xs: List[Int]): Int`

```
24 private val nonEmptyIntListGen: Gen[List[Int]] = Gen.nonEmptyListOf(Arbitrary.arbitrary[Int])

65 // -----
66 // maximum
67 // -----
68
69 property("maximum") = forAll(nonEmptyIntListGen) { (xs: List[Int]) =>
70   val result = max(xs)
71   xs.nonEmpty ==> xs.forall((x: Int) => x <= result)
72 }
```

This **nonEmptyIntListGen** *generator* creates arbitrary Lists with integers, creating random input for testing → Basically, it creates test case inputs for us ☺

SimpleFunctions ScalaCheck properties

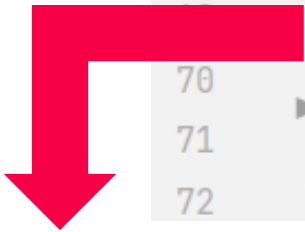
Example ScalaCheck property for `max(xs: List[Int]): Int`

```

24 private val nonEmptyIntListGen: Gen[List[Int]] = Gen.nonEmptyListOf(Arbitrary.arbitrary[Int])

65 // -----
66 // maximum
67 // -----
68
70 property("maximum") = forAll(nonEmptyIntListGen) { (xs: List[Int]) =>
71   val result = max(xs)
72   xs.nonEmpty ==> xs.forall((x: Int) => x <= result)
  }

```



This **property** goes through all elements `x` of the List `xs` and checks if `x` is the largest element

SimpleFunctions ScalaCheck properties

Example ScalaCheck property for **max(xs: List[Int]): Int**

```

24 private val nonEmptyIntListGen: Gen[List[Int]] = Gen.nonEmptyListOf(Arbitrary.arbitrary[Int])

65 // -----
66 // maximum
67 // -----
68
69 property("maximum") = forAll(nonEmptyIntListGen) { (xs: List[Int]) =>
70     val result = max(xs)
71     xs.nonEmpty ==> xs.forall((x: Int) => x <= result)
72 }

```

Combined with the generator: This ScalaCheck property generates test cases! ☺

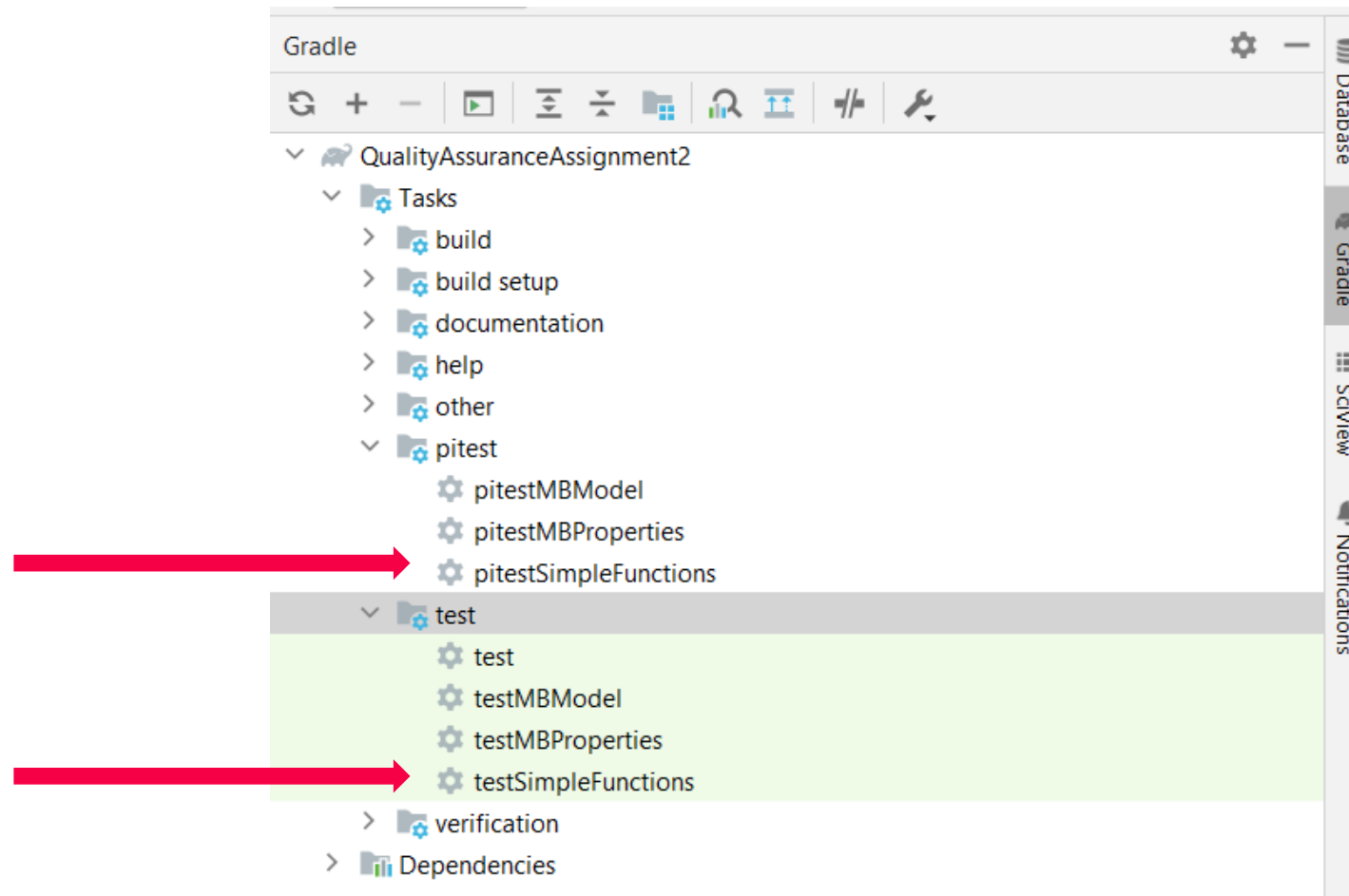
What we basically reached now:

"Don't write test cases,
generate them!"

(John Hughes)

SimpleFunctions ScalaCheck properties

Test your properties with Gradle



SimpleFunctions ScalaCheck properties

For this task 100% Line Coverage and Mutation Score of 92% is needed for full points

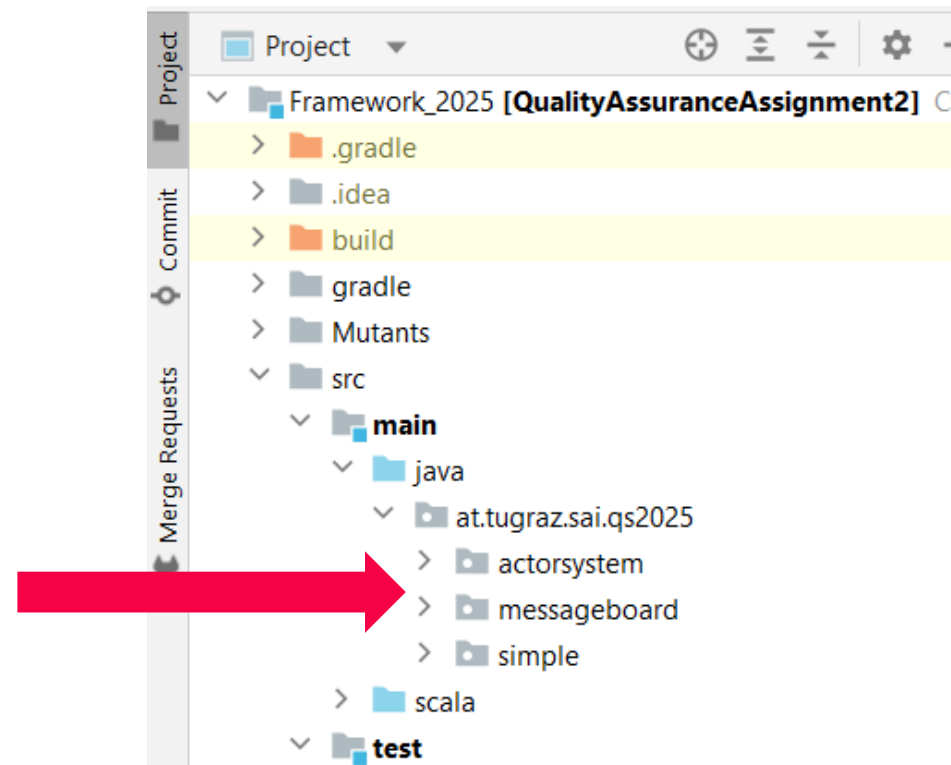
```

103 ▶ task pitestSimpleFunctions(type: MyPitestTask) {
104     coverageThreshold = 100
105     mutationThreshold = 92
106     reportDir = "pitestSimpleFunctions"
107     targetClasses = ['at.tugraz.sai.qs2025.simple.SimpleFunctions', 'at.tugraz.sai.qs2025.simple.SimpleFunctions$']
108     targetTests = ['at.tugraz.sai.qs2025.SimpleFunctionsTest']
109     mutators = PITEST_MUTATORS
110     group = "PiTest"
111     description = "Run PiTest with SimpleFunctions tests."
112 }
```

MessageBoard ScalaCheck properties

Apply your knowledge from the previous task to a bigger framework

Test the framework from Assignment 1 now with ScalaCheck properties:



MessageBoard ScalaCheck properties

Write properties for all 17 requirements from the assignment sheet

A generator for valid messages is already provided

(You might need new custom generators for all requirements)

A simple property for requirement R1 is also already provided in the framework (as seen on next slide)

MessageBoard ScalaCheck: Example Generator

```
14 // Generator for valid messages
15 val validMessageGen: Gen[String] = Gen.asciiPrintableStr.map(s =>
16   | if (s.length <= MAX_MESSAGE_LENGTH) s else s.substring(0, MAX_MESSAGE_LENGTH)
17   | )
```

MessageBoard ScalaCheck: Example R1 property

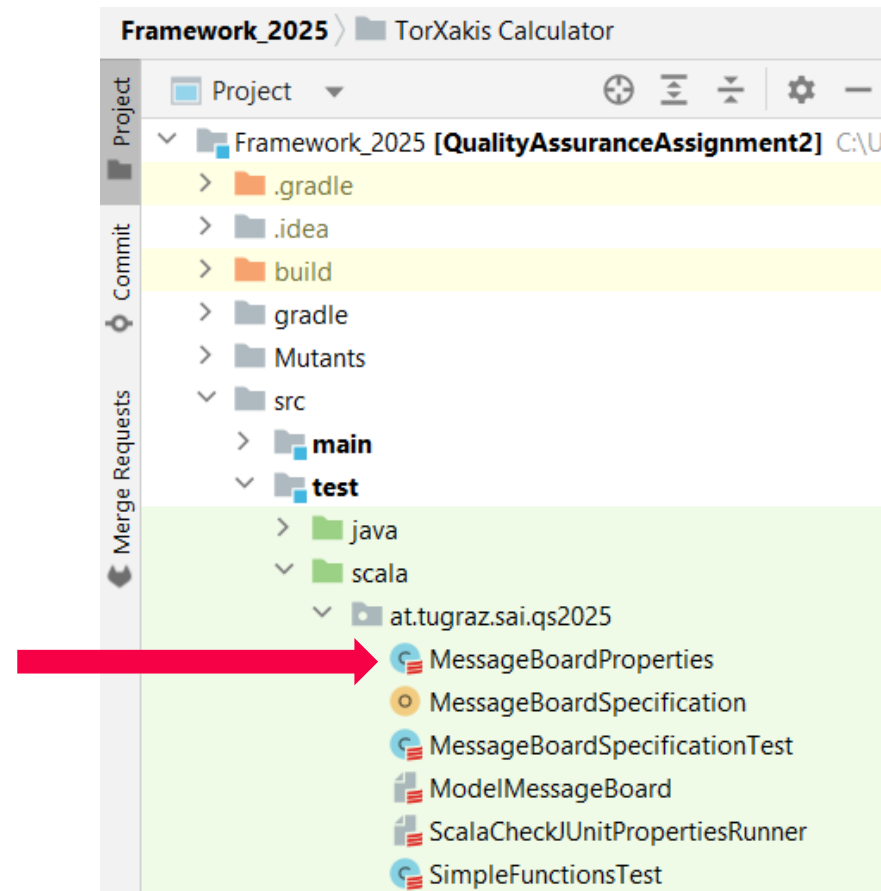
```

19 property("message length: Publish + Ack [R1]") = forAll { (author: String, message: String) =>
20   // arrange- initialize the message board
21   val sut = new SUTMessageBoard
22   sut.getDispatcher.tell(new InitCommunication(sut.getClient, sut.getCommId))
23   while (sut.getClient.receivedMessages.isEmpty)
24     sut.getSystem.runFor( numberOfTicks = 1)
25   val initAck = sut.getClient.receivedMessages.remove.asInstanceOf[InitAck]
26   val worker: SimulatedActor = initAck.worker
27
28   // act - send and receive the messages
29   worker.tell(new Publish(new UserMessage(author, message), sut.getCommId))
30   while (sut.getClient.receivedMessages.isEmpty)
31     sut.getSystem.runFor( numberOfTicks = 1)
32   val reply = sut.getClient.receivedMessages.remove()
33
34   worker.tell(new FinishCommunication(sut.getCommId))
35   while (sut.getClient.receivedMessages.isEmpty)
36     sut.getSystem.runFor( numberOfTicks = 1)
37   sut.getClient.receivedMessages.remove.asInstanceOf[FinishAck]
38
39   // assert - define your property and check against it
40   // The following classify is optional, it prints stats on the generated values.
41   // But the check inside is required.
42   classify(message.length <= MAX_MESSAGE_LENGTH, ifTrue = "valid message length", ifFalse = "invalid message length") {
43     // if operationAck is received, the message length should be smaller or equal to 10
44     reply.isInstanceOf[OperationAck] == message.length <= MAX_MESSAGE_LENGTH
45   }
46 }

```

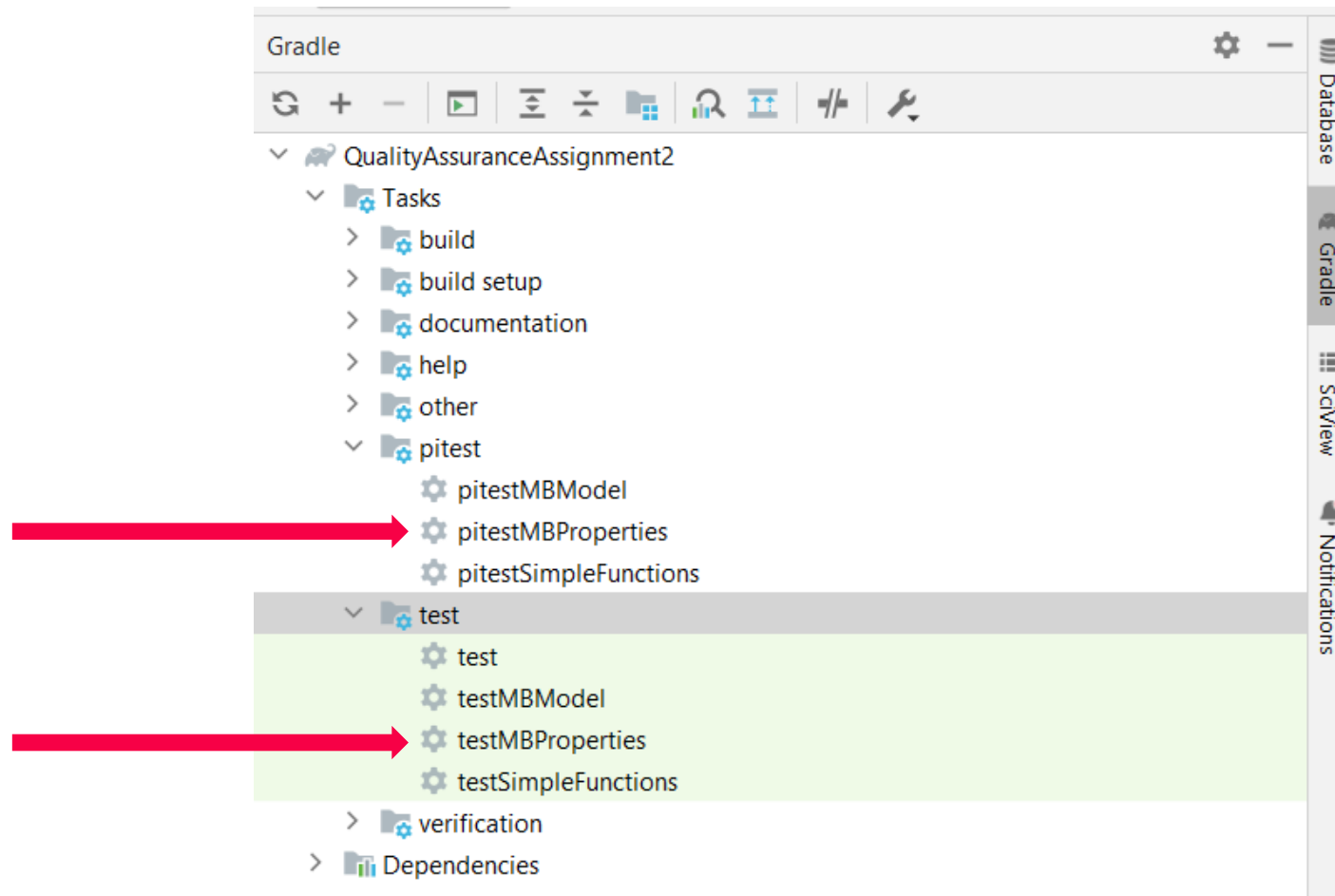
MessageBoard ScalaCheck properties

Write your properties in MessageBoardProperties.scala



MessageBoard ScalaCheck properties

Test your MB properties with Gradle:

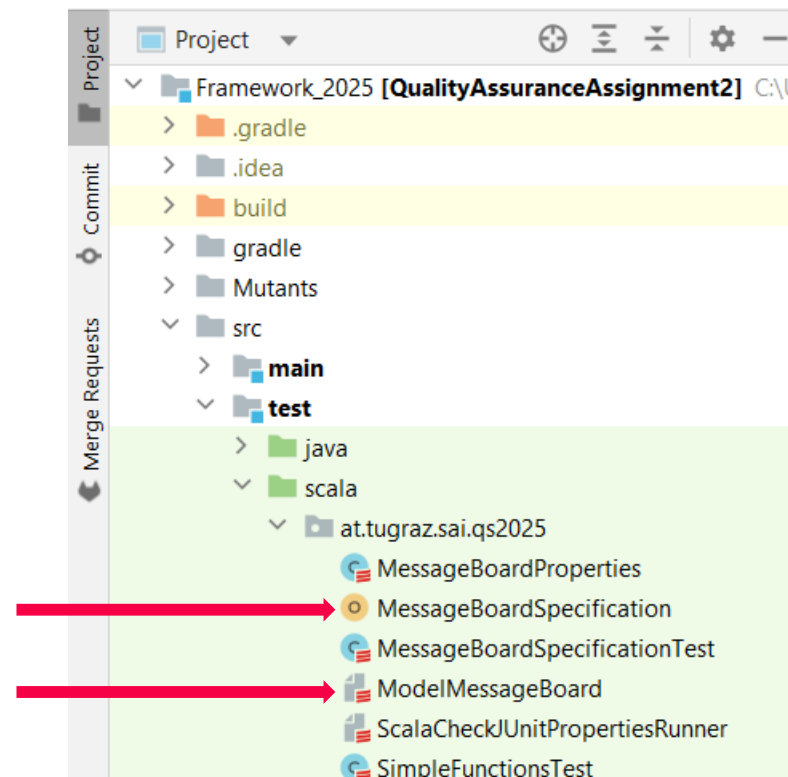


MessageBoard – Model-Based Testing

Implement a Scala Model (Specification) of the Message Board

The model used is in ModelMessageBoard.scala

The specification to be implemented is in MessageBoardSpecification.scala



MessageBoard – Model-Based Testing

The specification to be implemented is in `MessageBoardSpecification.scala`
From the assignment sheet:

- `run(sut: SUT): Result:`
Here you execute the command on the System-Under-Test (SUT). You should return a result that can be checked in the post-condition. We already suggested result types, but you are allowed to change them.
- `nextState(state: State): State:`
Here you execute the command on the model. Please pay attention: The model has to be immutable. You cannot modify the variables of the model directly; instead, you need to use `copy()` to return a modified instance.
- `postCondition(state: State, result: Try[Result]): Prop:`
Here you need to check if the behaviour of the SUT and the model is the same. Please pay attention: The state passed as the argument is the state **before** the invocation of `nextState()`, so you need to use `nextState(state)` to get the actual model after processing the command.

MessageBoard – Model-Based Testing

The specification to be implemented is in `MessageBoardSpecification.scala`

Modify for every Command the TODO stubs

Do NOT add preconditions (This will exhaust ScalaCheck's generators)

MessageBoard – Model-Based Testing

Test your Model:

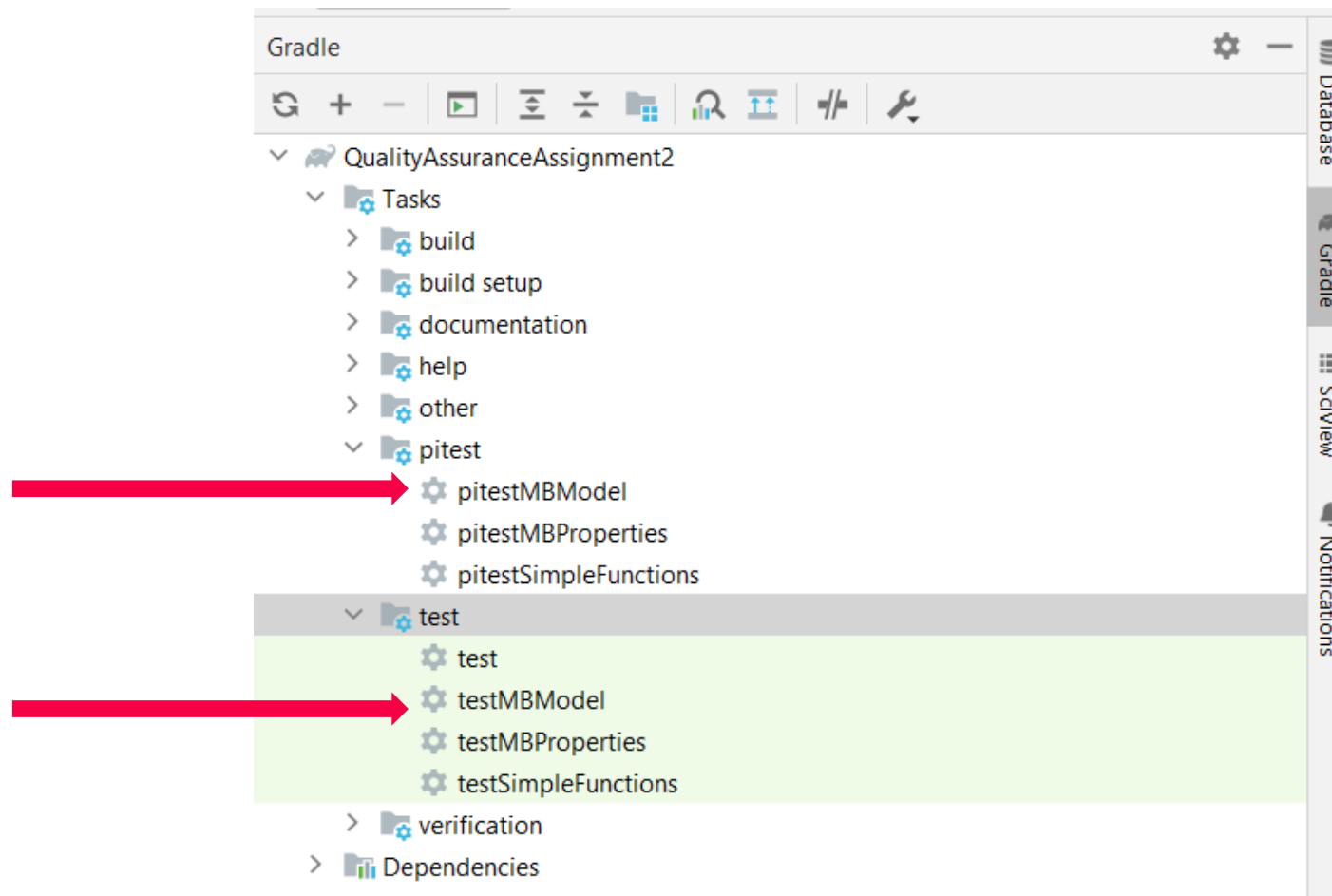
```

sebastian.albu
8  >> class MessageBoardSpecificationTest extends TestCase {
    sebastian.albu
9  > def testMessageBoardModelBased(): Unit = {
10   val params: Parameters => Parameters = p => p
11   .withTestCallback(ConsoleReporter(2))
12   .withMinSuccessfulTests( minSuccessfulTests = 300)
13   val result: Boolean = Test.check(MessageBoardSpecification.property())(params).passed
14   org.junit.Assert.assertTrue(result)
15  }
16 }

```

MessageBoard – Model-Based Testing

Test with Gradle:



MessageBoard – Kill Mutants (Python)

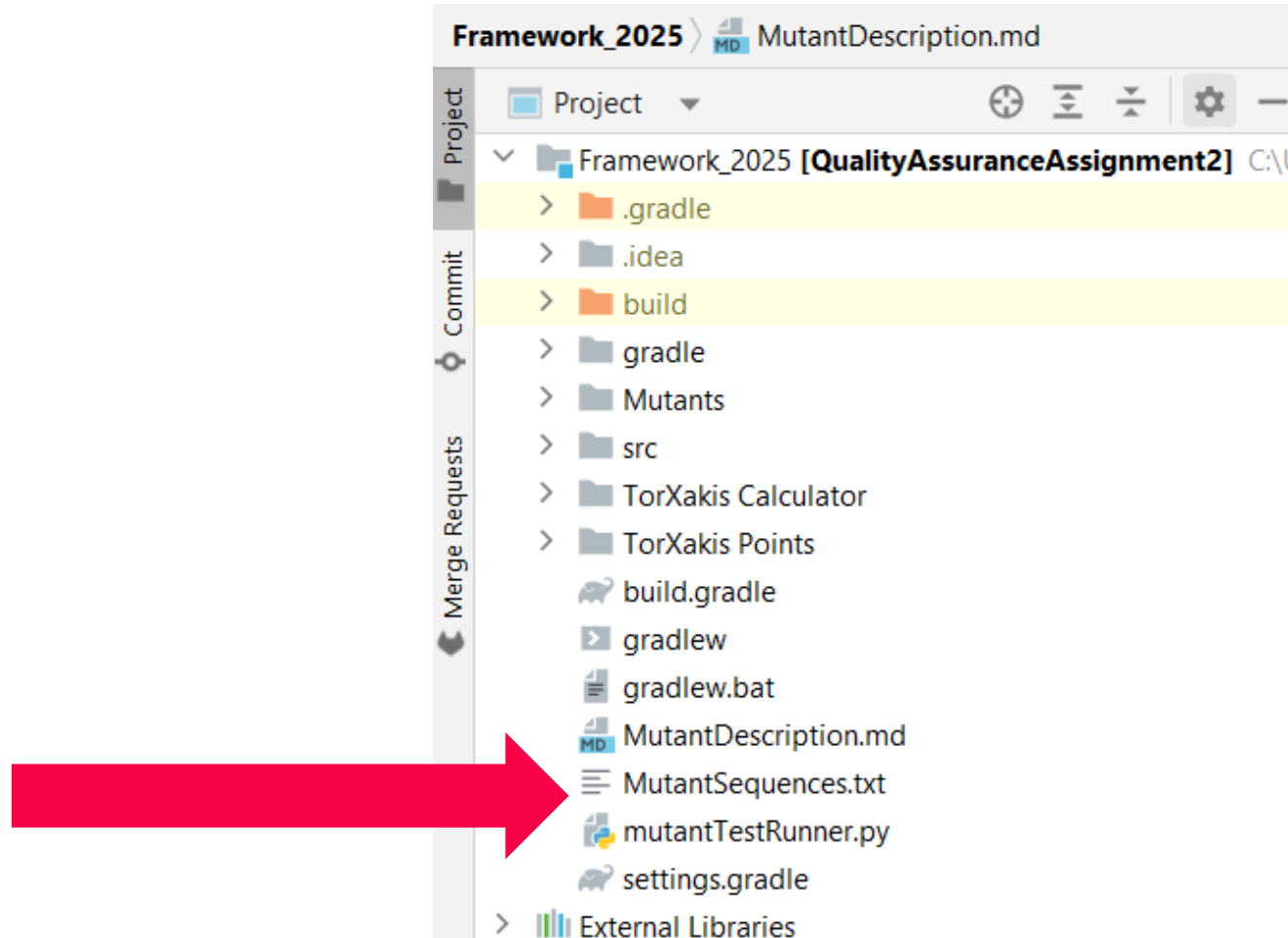
Your MessageBoardSpecification.scala is used

Mode-Based tests are run on each mutant.

Find sequences that identify mutants and provide them!

(More information in the assignment PDF)

MessageBoard – Kill Mutants (Python)



Bonus Task: Model-Based Testing with TorXakis



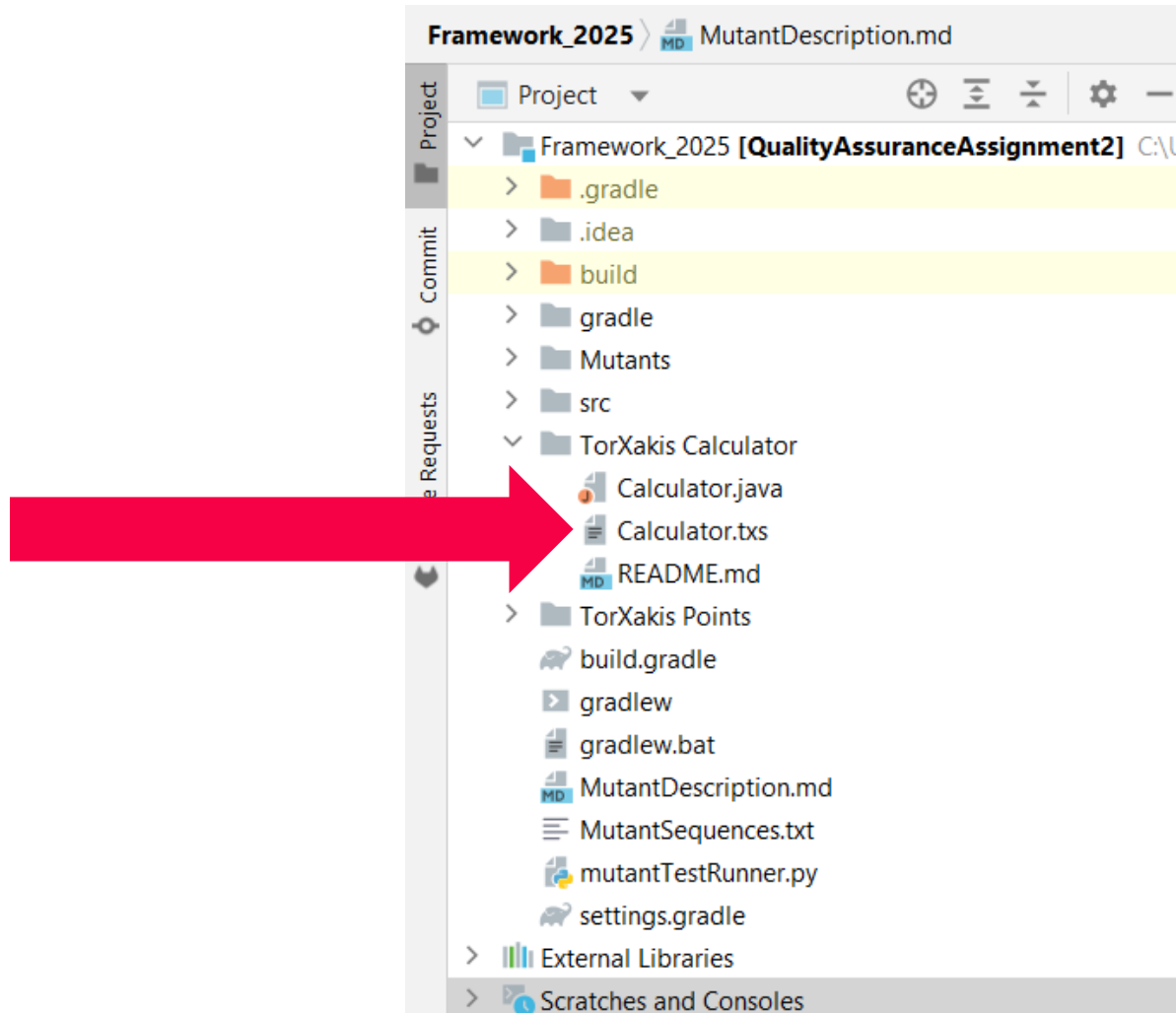
TorXakis is a MBT tester using Input-Output-Conformance (ioco)

You are provided a Java SUT of a Calculator

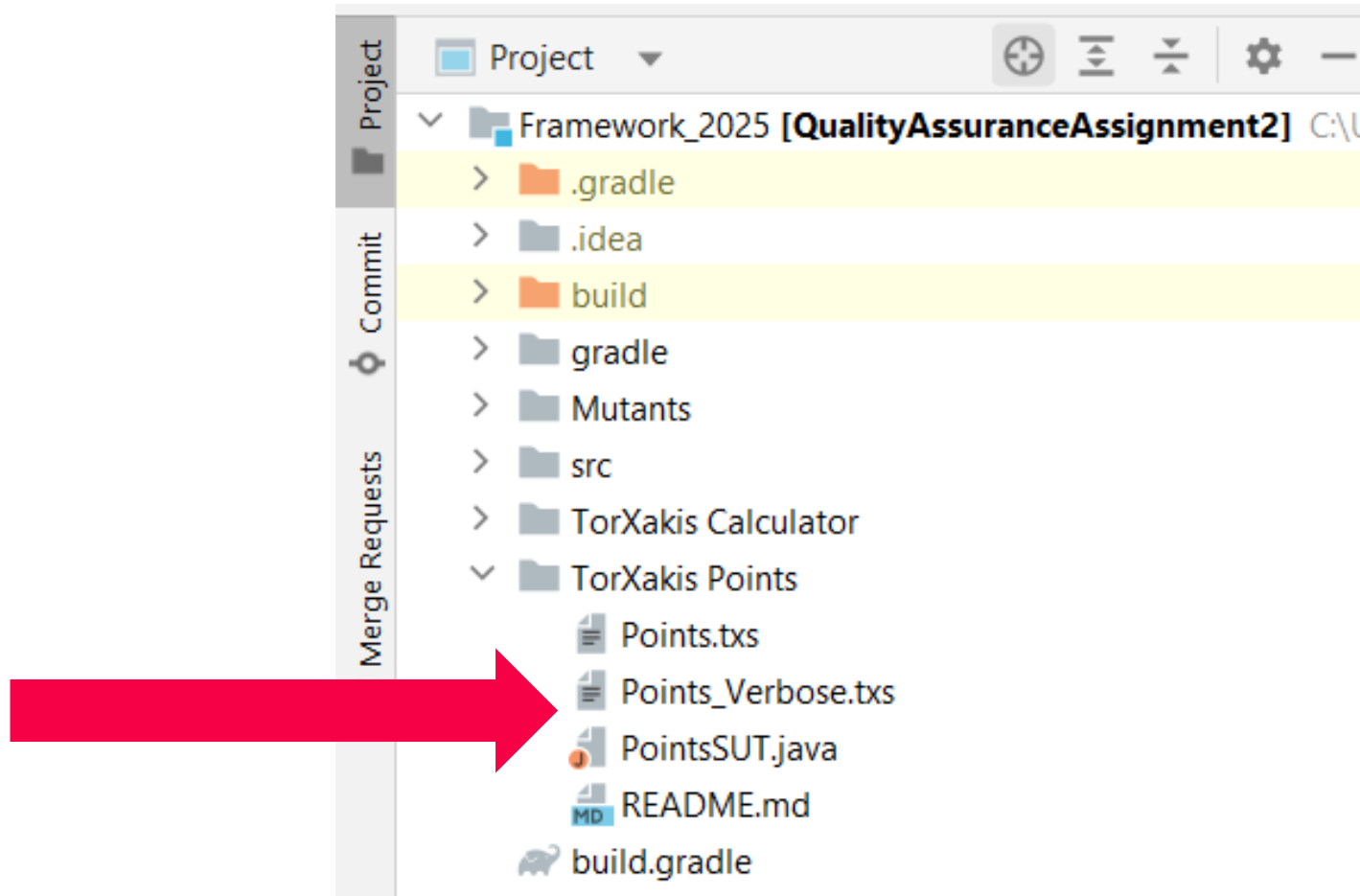
Write the Calculator.txs to perform MBT on the Java SUT

Follow the instructions in the README to see how it **should** behave!

Bonus Task: Model-Based Testing with TorXakis



Tutorial: Model-Based Testing with TorXakis



Model-Based Testing with TorXakis



TorXakis is a model-based testing framework designed for testing reactive systems.

It allows for specifying formal models of system behaviors, communication channels, and encoding/decoding methods.

It is especially used in conformance testing: verifying if an implementation behaves as expected by its specification. (ioco)

In this tutorial, we will go over the `Points_Verbose.txs` example 😊

Model-Based Testing with TorXakis



Modeling: Define the behavior of the system (processes, functions).

Channels: Specify how communication happens (input/output).

Encoding/Decoding: Specify how data is serialized for transmission.

System Under Test (SUT): The system tested against the model.

Model-Based Testing with TorXakis



TYPEDEF: Defines new types (e.g., structures).

FUNCDEF: Defines functions or predicates (e.g., validation or conditions).

PROCDEF: Defines a process or behavior (e.g., interaction logic).

CHANDEF: Defines communication channels.

MODELDEF: Combines everything into a complete model.

CNECTDEF: Defines how to connect to the SUT or a simulator.

Model-Based Testing with TorXakis



TYPEDEF: Defines new types (e.g., structures).

```
8
9  -- TYPEDEF defines a new type. Here, we define a "Point" type
10 -- that consists of two integer fields: x and y.
11
12 TYPEDEF Point ::= Point { x, y :: Int } ENDEF
13
14     -- "Point" is a custom type, and it's structured by two fields, "x" and "y", which are both of type Int.
15
```

Model-Based Testing with TorXakis



FUNCDEF: Defines functions or predicates (e.g., validation or conditions).

```

26  -- FUNCDEF for "inRange": Determines whether p2 is within a defined range of p1.
27  -- The conditions are:
28  -- (i) x(p1) < x(p2) and y(p1) > y(p2), OR
29  -- (ii) x(p1) > x(p2) and y(p1) < y(p2).
30
31  FUNCDEF inRange (p1, p2 :: Point) :: Bool ::=
32  | (( x(p1) < x(p2) ) /\ ( y(p1) > y(p2) ))
33  | \/ (( x(p1) > x(p2) ) /\ ( y(p1) < y(p2) ))
34  ENDEF
35
36  -- This function "inRange" checks if p2 lies within a specific range of p1, with two possible conditions connected by "OR" (\/).
37  -- It checks:
38  --   - (x(p1) < x(p2) AND y(p1) > y(p2)) OR
39  --   - (x(p1) > x(p2) AND y(p1) < y(p2))
40

```

Model-Based Testing with TorXakis



FUNCDEF: Defines functions or predicates (e.g., validation or conditions).

```

16      -- FUNCDEF defines a function. The function "validPoint" takes a Point (p)
17      -- and returns a Bool indicating whether both x and y are nonzero.
18
19      FUNCDEF validPoint (p :: Point) :: Bool ::=
20      |   ( x(p) <> 0 ) /\ ( y(p) <> 0 )
21      ENDDDEF
22
23      -- FUNCDEF introduces a function. Here, "validPoint" checks if the "x" and "y" values of the point are not equal to 0.
24      -- The syntax "x(p)" retrieves the x-coordinate of the point p, similar fashion then for "y(p)".

```

Model-Based Testing with TorXakis



PROCDEF: Defines a process or behavior (e.g., interaction logic).

```

40
41 -- PROCDEF defines a process behavior.
42 -- "allowedBehaviour" models a process where:
43 -- 1. The system receives a point "p1" through channel "In"
44 --    (only if validPoint(p1) holds).
45 -- 2. It then receives another point "p2" through channel "Out"
46 --    (only if validPoint(p2) and inRange(p1, p2) hold).
47 -- 3. This process repeats indefinitely.
48
49 PROCDEF allowedBehaviour [ In, Out :: Point ] () ::=
50     In ? p1 [[ validPoint(p1) ]] -- The process receives a point from channel "In", named "p1".
51     >-> Out ? p2 [[ validPoint(p2) /\ inRange (p1,p2) ]] -- The process sends a point through "Out" named "p2" (with checks on its validity).
52     >-> allowedBehaviour [In,Out] () -- The process recursively calls itself to continue the cycle.
53 ENDEF
54
55 -- PROCDEF defines the behavior of a process. "allowedBehaviour" represents an interaction sequence where:
56 -- - "In ? p1" means receiving a point "p1" from the input channel "In".
57 -- - "[[ validPoint(p1) ]]" means that the system will only accept "p1" if it satisfies the "validPoint" function.
58 -- - "Out ? p2" means receiving a point "p2" from the output channel "Out".
59 -- - "[[ validPoint(p2) /\ inRange(p1, p2) ]]" means "p2" must also satisfy "validPoint" and be within the range defined by "p1".
60 -- - ">->" indicates sequential execution: after receiving and checking "p1", the process continues to receive "p2".
61 -- - "allowedBehaviour [In,Out] ()" indicates the process repeats indefinitely.
62

```

Model-Based Testing with TorXakis



CHANDEF: Defines communication channels.

```

63      -- CHANDEF defines communication channels.
64      -- "In" and "Out" channels transmit messages of type "Point".
65
66      CHANDEF Channels ::=
67      |   In, Out :: Point
68      ENNDEF
69
70      -- CHANDEF defines communication channels in the system:
71      -- - "In" and "Out" are declared as channels that transmit messages of type "Point".
72

```

Model-Based Testing with TorXakis



MODELDEF: Combines everything into a complete model.

```

73  -- MODELDEF defines the full model.
74  -- This model:
75  -- - Declares "In" as an input channel.
76  -- - Declares "Out" as an output channel.
77  -- - Specifies "allowedBehaviour" as the behavior of the model.
78
79  MODELDEF Model ::=
80      |CHAN IN      In
81      |CHAN OUT     Out
82      |BEHAVIOUR    allowedBehaviour [ In, Out ] ()
83  ENDEF
84
85  -- MODELDEF defines the full model:
86  -- - "CHAN IN In" declares the "In" channel as an input channel in the model.
87  -- - "CHAN OUT Out" declares the "Out" channel as an output channel in the model.
88  -- - "BEHAVIOUR allowedBehaviour [ In, Out ] ()" specifies that the "allowedBehaviour" process is the model's behavior,
89  -- where "In" and "Out" are the communication channels.
90

```

Model-Based Testing with TorXakis



CNECTDEF: Defines how to connect to the SUT or a simulator.

```

93  -- CNECTDEF defines a connection to the System Under Test (SUT).
94  -- "Sut" acts as a CLIENT, sending and receiving messages over a socket.
95
96  CNECTDEF Sut ::=
97      CLIENTSOCK
98      -- CHAN OUT defines an outgoing channel for sending input to the SUT.
99      CHAN OUT In          HOST "localhost" PORT 9999
100     -- ENCODE specifies how to serialize messages before sending.
101     ENCODE In ? s         -> ! toString(s)
102     -- CHAN IN defines an incoming channel for receiving responses from the SUT.
103     CHAN IN Out          HOST "localhost" PORT 9999
104     -- DECODE specifies how to deserialize messages upon receiving.
105     DECODE Out ! fromString(s) <- ? s
106  ENNDEF
107
108  -- CNECTDEF defines a connection to the System Under Test (SUT):
109  -- - "CLIENTSOCK" means that this is a client-side connection.
110  -- - "CHAN OUT In" defines the outgoing channel for sending data to the SUT over a socket connection to "localhost" on port 9999.
111  -- - "ENCODE In ? s -> ! toString(s)" means that when the point "In" is received, it is converted to a string using the function "toString".
112  -- - "CHAN IN Out" defines the incoming channel for receiving data from the SUT.
113  -- - "DECODE Out ! fromString(s) <- ? s" means when data is received from the "Out" channel, it is decoded using "fromString".
114  -- "Sim" acts as a SERVER that simulates the expected behavior.
115  -- This is useful for testing without an actual SUT.

```

Model-Based Testing with TorXakis



CNECTDEF: Defines how to connect to the SUT or a simulator.

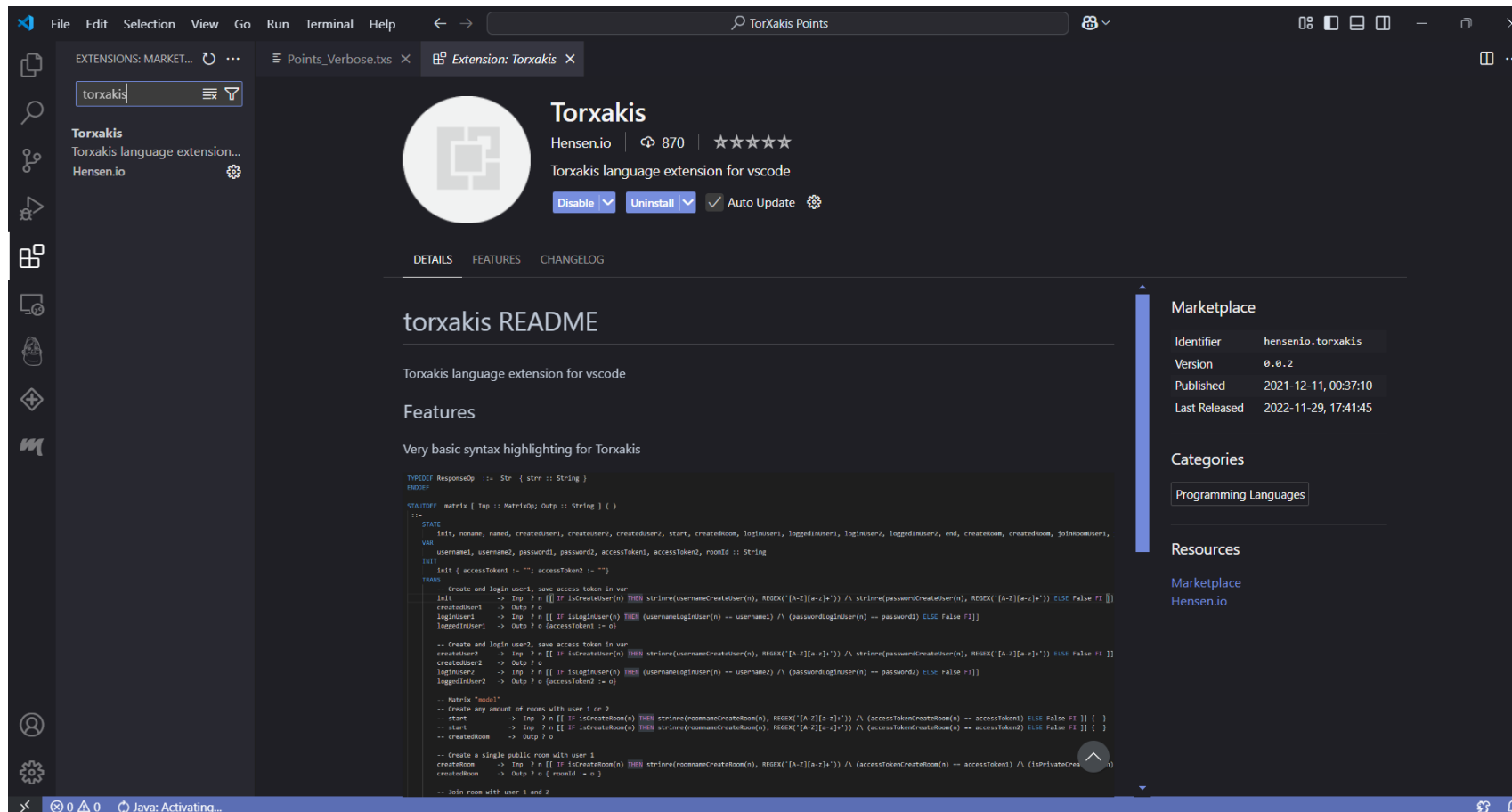
```

117 CNECTDEF Sim ::=
118     SERVERSOCK
119     -- Outgoing channel for responses.
120     CHAN OUT Out          HOST "localhost" PORT 9999
121     ENCODE Out ? s        -> ! toString(s)
122     -- Incoming channel for requests.
123     CHAN IN In           HOST "localhost" PORT 9999
124     DECODE In ! fromString(s) <- ? s
125 ENDDEF
126
127 -- CNECTDEF defines a server-side connection ("Sim") that simulates the expected behavior:
128 -- - "SERVERSOCK" indicates this is a server-side connection.
129 -- - "CHAN OUT Out" is the outgoing channel used by the server to send responses.
130 -- - "ENCODE Out ? s -> ! toString(s)" means that when data is received from the "Out" channel, it is encoded to a string.
131 -- - "CHAN IN In" is the incoming channel used by the server to receive requests.
132 -- - "DECODE In ! fromString(s) <- ? s" means that the data received from the "In" channel is decoded from a string.
133

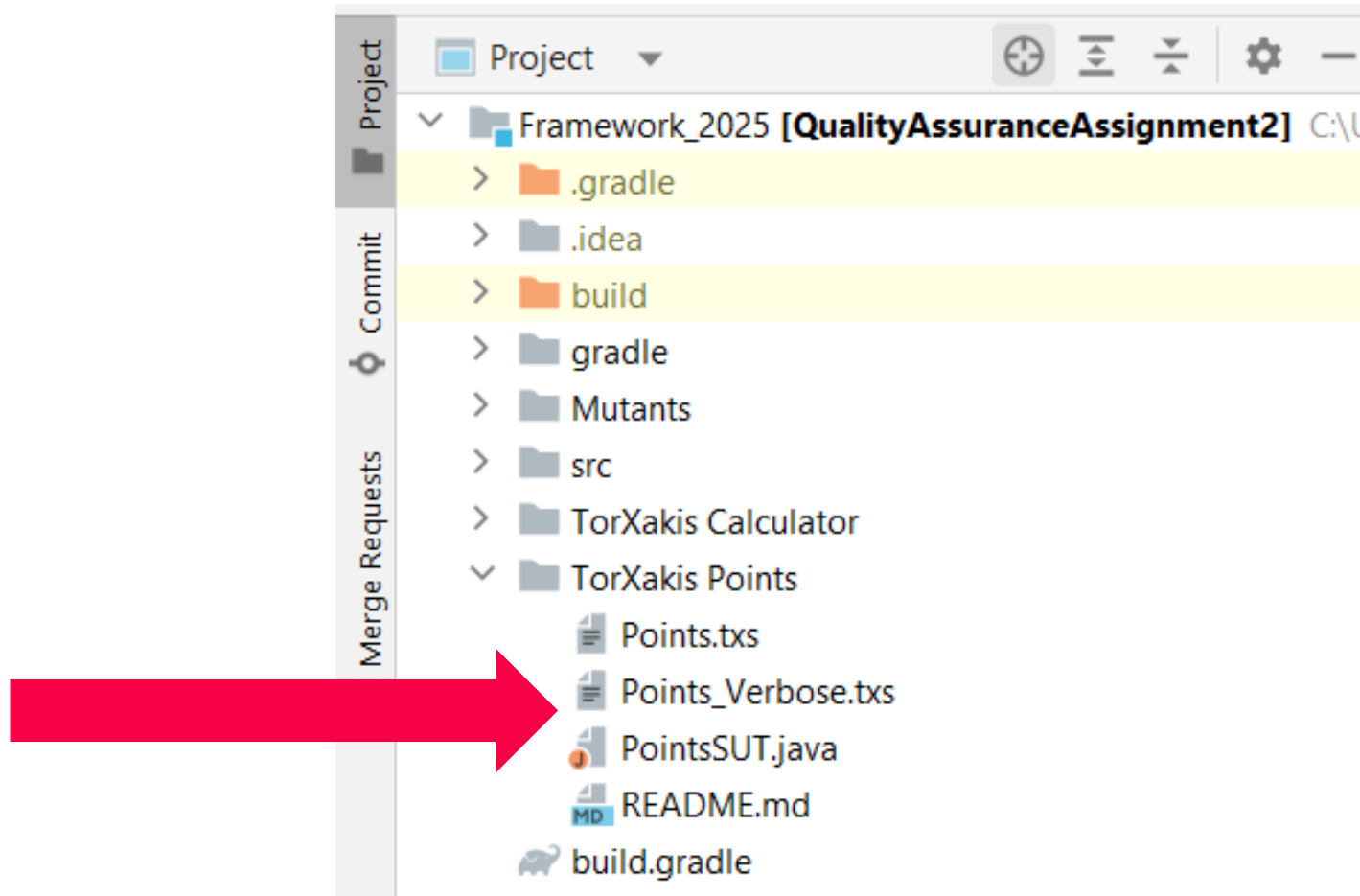
```

Bonus Task: Model-Based Testing with TorXakis

Helpful when working in VS Code:



Tutorial: Try it out yourself (README) 😊



Quality Assurance in SW- Development (Practicals)

Assignment 2 “Tutorial” – ScalaCheck, TorXakis

Summer 2025